

File, Input, and Output

Computer Files

A **computer file** is a collection of data stored on a nonvolatile device or a permanent storage device such as a hard disk, universal serial bus (USB) drive, compact disk, and reel or cassette of magnetic tape. Additionally, although contents vary, files have many common characteristics, including their size, which specifies the space on a disk section or storage device, their names, and their specific time of creation.

Files can be categorized by the way they store data:

- **Text files** contain data that can be read in a text editor as the data is encoded using a scheme (ASCII or Unicode). These can be **data files** that contain facts and figures, including a payroll file with employee numbers, names, and salaries. These can also be **program files** or **application files** that store software instructions.
- **Binary files** contain data items that have not been encoded as text. A binary file's contents are in binary format (0s and 1s), which is not easily understood by viewing them in a text editor. Examples include music, images, and the compiled program files with a `.class` extension.

A **root directory** or main directory of the storage device can be used to store a permanent file. For better organization, most computer users organize their files by placing each into a **folder** or **directory**. Users can create folders within folders to form a hierarchy. A complete list of the disk drive and the hierarchy of directories in which a file resides is called its **path**.

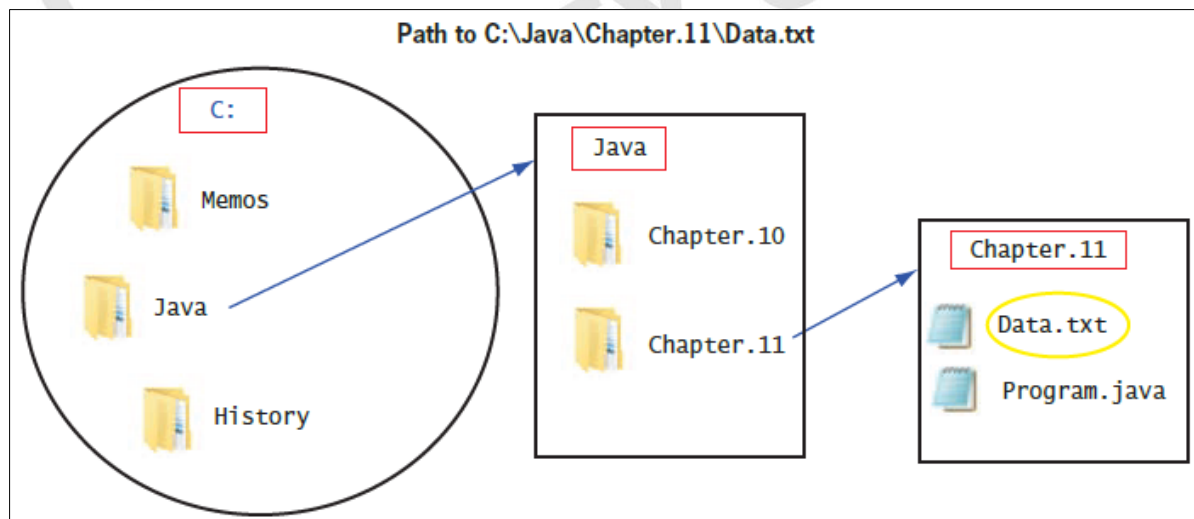


Figure 1. Path location example. Retrieved from Farrell, J. (2023). *Java programming, 10th edition*. Cengage.

The illustration shows the complete path for a Windows file named `Data.txt`, saved on the C drive in a folder named `Chapter.11`, which is within a folder named `Java`. The **backslash** (`\`) in the Windows operating system is the **path identifier** which separates the path components. A **slash** (`/`) is used as the delimiter in the Solaris (UNIX) operating system.

The following tasks are typically performed when working with stored files in an application:

- Determining whether and where a path or file exists
- Opening a file
- Writing to a file
- Reading from a file
- Closing a file
- Deleting a file

Java provides the `Path` and `Files` classes to work with stored files.

The Path and Files Classes

The `Path` class creates objects containing information about files and directories, including their locations, sizes, creation dates, and whether they exist. On the other hand, the `Files` class performs operations on files and directories, including deletion, determining their attributes, and creating input and output streams.

The `import java.nio.file.*;` statement can be included in a Java program to use both the `Path` and `Files` classes. The *nio* in `java.nio` stands for *new input/output* as its classes are “new”. **Creating a Path**

Creating a `Path` can be initiated by determining the file system on the host computer by using a statement such as the following:

```
FileSystem fs = FileSystems.getDefault();
```

This statement creates a `FileSystem` object using the `getDefault()` method in the `FileSystems` class.

It uses two (2) different classes: `FileSystem` class, without an ending *s*, to instantiate the object, and the `FileSystems` class, with an ending *s*, which contains factory methods. In object-oriented programming (OOP), a **factory** is an object that creates other objects.

After a `FileSystem` object is created, a `Path` can be defined using the `getPath()` method with it:

```
Path filePath = fs.getPath("C:\\Java\\Chapter.11\\Data.txt");
```

As mentioned, a backslash (`\`) is used as an escape sequence in Java. To enter it as a path delimiter within a string, two (2) backslashes should be typed to indicate a single backslash.

Another way to create a `Path` is using the `Paths` class (with an ending *s*). The `Paths` class is considered a helper class that eliminates the need to create a `FileSystem` object. The `get()` method of this class calls the `getPath()` method of the default file system without requiring a `FileSystem` object. A `Path` object can be created using the following statement:

```
Path filePath = Paths.get("C:\\Java\\Chapter.11\\SampleFile.txt");
```

After the `Path` is created, use its identifier, `filePath` in this case, to refer to the file and perform operations on it. The `C:\Java\Chapter.11\SampleFile.txt` is the full name of a stored file when the operating system refers to it, but the path is known as `filePath` within the application.

Additionally, a path can either be absolute or relative.

An **absolute path** is a complete path that does not need any information to locate a file on a system. For example, C:\\Java\\Chapter.11\\SampleFile.txt is an absolute path.

A **relative path** depends on other path information. For example, a simple path such as SampleFile.txt is relative. When working with a path containing only a filename, the file is assumed to be in the same folder as the program using it. In context, referring to a relative path such as Chapter.11\\SampleFile.txt, the Chapter.11 folder is considered a subfolder of the current directory, and the SampleFile.txt can be within this folder.

Converting a Relative Path to an Absolute Path

To convert a relative path to an absolute path, the `toAbsolutePath()` method is used.

The program, PathDemo2, below shows a program that asks a user for a filename and converts it to an absolute path, if necessary.

```
import java.util.Scanner;
import java.nio.file.*;
public class PathDemo2 {
    public static void main(String[] args) {
        String name;
        Scanner keyboard = new Scanner(System.in);
        System.out.print("Enter a file name >> ");
        name = keyboard.nextLine();
        Path inputPath = Paths.get(name);
        Path fullPath = inputPath.toAbsolutePath();
        System.out.println("Full path is " + fullPath.toString());
    }
}
```

When the PathDemo2 program executes and the filename represents an absolute Path, the program will not change the input. But, if the input represents a relative Path, this program creates an absolute path by assigning the file to the current directory.

The execution of this program:

```
Enter a file name >> Memo.txt
Full path is C:\\Java\\Memo.txt
```

Retrieving Path Information

The following table summarizes several useful Path methods.

Method	Description
String toString()	Returns the String representation of the path, eliminating double backslashes. Also, the toString() method is overridden from the Object class.
Path getFileName()	Returns the file or directory denoted by this Path; the last item in the sequence of name elements. This is frequently a filename but also a folder name.
int getNameCount()	Returns the number of name elements in the Path
Path getName(int)	Returns the name in the position of the Path specified by the integer parameter

A Path's elements are accessed using an index. The top-level element in the directory structure is located at index 0, while the lowest element in the structure is accessed by the `getName()` method. This has an index of one (1) less than the number of items on the list.

The `getNameCount()` method retrieves the number of names in the list, while the `getName(int)` method retrieves the name in the position specified by the argument.

The program, `PathDemo` class, below shows a demonstration program that creates a `Path` and uses some of the mentioned methods.

```
import java.nio.file.*;
public class PathDemo {
    public static void main(String[] args) {
        Path filePath = Paths.get("C:\\Java\\Chapter.11\\Data.txt");
        int count = filePath.getNameCount();
        System.out.println("Path is " + filePath.toString());
        System.out.println("File name is " + filePath.getFileName());
        System.out.println("There are " + count + " elements in the file path");
        for(int x = 0; x < count; ++x)
            System.out.println("Element " + x + " is " + filePath.getName(x));
    }
}
```

The execution of this program:

```
Path is C:\Java\Chapter.11\Data.txt
File name is Data.txt
There are 3 elements in the file path
Element 0 is Java
Element 1 is Chapter.11
Element 2 is Data.txt
```

Checking File Accessibility

The `checkAccess()` method is used to verify that a file exists and that the program can access it as needed.

The following import statement accesses constants that can be used as arguments to the method:

```
import static java.nio.file.AccessMode.*;
```

Assuming a `Path` named `filePath` has been declared, the syntax to use with `checkAccess()` is as follows:

```
filePath.getFileSystem().provider().checkAccess();
```

Furthermore, the following can be used as arguments to the `checkAccess()` methods:

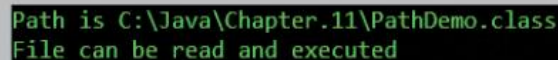
- **No argument** – checks that the file exists. As an alternative to using `checkAccess()` with no argument to determine whether a file exists, the `Files.exists()` method can be used as a substitute and pass it a `Path` argument.
- **READ** – checks that the file exists and the program has permission to read the file
- **WRITE** – checks that the file exists and the program has permission to write to the file
- **EXECUTE** – checks that the file exists and the program has permission to execute the file

Multiple arguments, separated by commas, can be used in the `checkAccess()` method. Additionally, an `IOException` is thrown if the file named in the method call cannot be accessed.

The program, PathDemo3 class, below shows an application that declares a Path and checks whether a file named there can be both read and executed. Notice that the java.io.IOException package must also be imported.

```
import java.nio.file.*;
import static java.nio.file.AccessMode.*;
import java.io.IOException;
public class PathDemo3 {
    public static void main(String[] args) {
        Path filePath = Paths.get("C:\\Java\\Chapter.11\\PathDemo.class");
        System.out.println("Path is " + filePath.toString());
        try {
            filePath.getFileSystem().provider().checkAccess(filePath, READ, EXECUTE);
            System.out.println("File can be read and executed");
        }
        catch(IOException e) {
            System.out.println("File cannot be used for this application");
        }
    }
}
```

The execution of this program:



```
Path is C:\Java\Chapter.11\PathDemo.class
File can be read and executed
```

Determining File Attributes

The readAttributes() method of the Files class can retrieve useful information about a file. This takes two (2) arguments: a Path object and BasicFileAttributes.class. It returns an instance of the BasicFileAttributes class. An instance with a statement can be created such as the following:

```
BasicFileAttributes attr = Files.readAttributes(filePath, BasicFileAttributes.class);
```

After a BasicFileAttributes object is created, several methods for retrieving information about a file can be used. For example, the size() method to return the size of a file in bytes or methods such as creationTime() and lastModifiedTime() to return relevant file times.

The program, PathDemo4 class, below contains these methods.

```
import java.nio.file.*;
import java.nio.file.attribute.*;
import java.io.IOException;
public class PathDemo4 {
    public static void main(String[] args) {
        Path filePath = Paths.get("C:\\Java\\Chapter.11\\Data.txt");
        try {
            BasicFileAttributes attr =
                Files.readAttributes(filePath, BasicFileAttributes.class);
            System.out.println("Creation time " + attr.creationTime());
            System.out.println("Last modified time " + attr.lastModifiedTime());
            System.out.println("Size " + attr.size());
        }
        catch(IOException e) {
```

```

        System.out.println("IO Exception");
    }
}

```

The time methods in the PathDemo4 program return a `FileTime` object converted to a `String` in the `println()` method calls. `FileTime` objects are represented in the format: `yyyy-mm-ddThh:mm:ss`

The execution of this program:

```

Creation time 2024-04-27T17:40:44.3320447Z
Last modified time 2024-04-28T17:49:30.4969912Z
Size 18

```

In a `FileTime` object, the four-digit year is followed by a hyphen (-), the two-digit month, another hyphen, and the two-digit day. Following a *T* for *Time*, the hour, minute, and seconds, even fractions of a second, are separated by colons (:).

The IO Classes

The illustration below shows a partial hierarchical relationship of some of the classes Java uses for input and output (IO) operations.

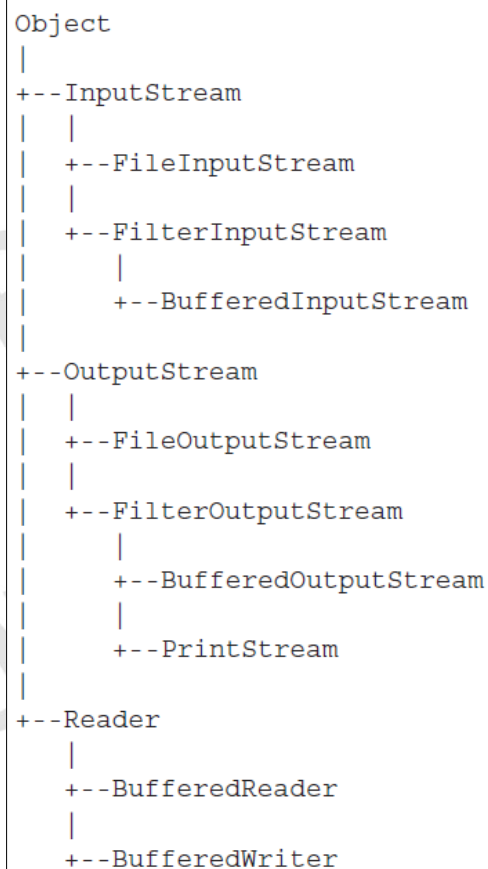


Figure 2. IO operations. Retrieved from Farrell, J. (2023). *Java programming, 10th edition*. Cengage.

The hierarchy shows that `InputStream`, `OutputStream`, and `Reader` are subclasses of the `Object` class.

The table below lists the capabilities and descriptions of selected classes used for input and output.

Classes	Description
InputStream	An abstract class that contains methods for performing input
FileInputStream	Child of InputStream that provides the capability to read from disk files
BufferedInputStream	Child of FilterInputStream that handles input from a system's standard (or default) input device, usually the keyboard
OutputStream	An abstract class that contains methods for performing output
FileOutputStream	Child of OutputStream that allows writing to disk files
BufferedOutputStream	Child of FilterOutputStream that handles output from a system's standard (or default) output device, usually the monitor
PrintStream	Child of FilterOutputStream. Java's System class contains a PrintStream object named System.out
Reader	Abstract class for reading character streams; the only methods that a subclass must implement are read(char[], int, int) and close()
BufferedReader	Reads text from a character-input stream, buffering characters to provide for efficient reading of characters, arrays, and lines
BufferedWriter	Writes text to a character-output stream, buffering characters to provide for the efficient writing of characters, arrays, and lines

The table below lists some of the OutputStream methods.

OutputStream Methods	Description
void close()	Closes the output stream and releases any system resources associated with the stream
void flush()	Flushes the output stream; if any bytes are buffered, they will be written
void write(byte[] b)	Writes all the bytes to the output stream from the specified byte array
void write(byte[] b, int off, int len)	Writes bytes to the output stream from the specified byte array starting at offset position off for a length of len characters

The program, ScreenOut class shows an application declaring a String of letter grades allowed in a course.

```
import java.io.*;
public class ScreenOut {
    public static void main(String[] args) {
        String s = "ABCD";
        byte[] data = s.getBytes();
        OutputStream output = null;
        try {
            output = System.out;
            output.write(data);
            output.flush();
            output.close();
        }
        catch(Exception e) {
            System.out.println("Message: " + e);
        }
    }
}
```

Then, the `getBytes()` method converts the `String` to an array of bytes. An `OutputStream` object is declared, and `System.out` is assigned to the `OutputStream` reference in a try block. Lastly, the `write()` method accepts the byte array and sends it to the output device, and then the output stream is flushed and closed.

The execution of this program:

**Reference:**

Farrell, J. (2023). *Java programming, 10th edition*. Cengage.